

IT3042 — LẬP TRÌNH BACKEND LARAVEL

HƯỚNG DẪN THỰC HÀNH — BUỔI 6

ELOQUENT ORM — MODEL & CRUD

Khoa Công nghệ Thông tin — Trường Đại học Phú Xuân

3 BÀI LAB · TỔNG THỜI LƯỢNG ~120 PHÚT · DỰ ÁN: PHU-XUAN-BLOG

Lab	Nội dung — Thời lượng — Mức độ
Lab 1	Tạo Model Post & cấu hình \$fillable/\$casts, kiểm tra bằng Tinker · 30 phút · Cơ bản
Lab 2	Refactor toàn bộ PostController sang Eloquent (CRUD hoàn chỉnh) · 60 phút · Trung bình
Lab 3	Soft Delete cho Post + trang khôi phục + cài Laravel Debugbar · 30 phút · Nâng cao / Tự chọn

CHUẨN BỊ

Trước khi bắt đầu — checklist chuẩn bị:

- Project phu-xuan-blog chạy được bình thường: php artisan serve không lỗi.
- Đã hoàn thành migration Buổi 5: bảng posts, categories, tags, post_tag, comments đã tồn tại trong database (kiểm tra bằng php artisan migrate:status).
- PostController, routes/web.php và các view (index/create/edit/show) từ Buổi 1–4 đã có sẵn (hiện đang dùng DB::table() hoặc mảng dữ liệu mẫu).
- Mở sẵn 2 cửa sổ terminal: một để chạy artisan serve, một để chạy các lệnh artisan khác.

Cấu trúc file sẽ tạo / chỉnh sửa trong buổi

```
phu-xuan-blog/
├── app/
│   ├── Http/Controllers/
│   │   └── PostController.php          ← refactor toàn bộ (Lab 2)
│   ├── Models/
│   │   └── Post.php                  ← tạo mới (Lab 1), thêm SoftDeletes (Lab 3)
│   └── 3)
├── database/migrations/
│   └── ..._add_deleted_at_to_posts_table.php ← tạo mới (Lab 3)
├── resources/views/posts/
│   ├── index.blade.php              ← cập nhật cú pháp object (Lab 2)
│   ├── trashed.blade.php           ← tạo mới (Lab 3)
│   └── ...
├── routes/
└── web.php                          ← thêm route trashed/restore (Lab 3)
```


1. LAB 1 — TẠO MODEL POST & CẤU HÌNH

Mục tiêu: tạo Eloquent Model đại diện cho bảng posts, khai báo đúng \$fillable/\$casts theo schema Buổi 5, và xác nhận Model hoạt động đúng trước khi gắn vào Controller.

Bước 1 — Tạo Model bằng Artisan

Mở terminal tại thư mục gốc project, chạy lệnh sau:

```
php artisan make:model Post
```

CHECKPOINT

Kết quả mong đợi trong terminal: “Model [app/Models/Post.php] created successfully.”

File app/Models/Post.php xuất hiện, lúc này class Post chỉ extends Model, chưa có cấu hình gì thêm.

Bước 2 — Khai báo \$fillable

Mở app/Models/Post.php, thêm thuộc tính \$fillable đúng theo các cột đã thiết kế ở Buổi 5 (không bao gồm id, created_at, updated_at — các cột này Laravel tự quản lý):

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Post extends Model
{
    protected $fillable = [
        'title',
        'slug',
        'body',
        'category_id',
        'user_id',
        'published_at',
        'is_featured',
    ];
}
```

LƯU Ý

Nếu bảng posts của bạn ở Buổi 5 đặt tên cột khác (ví dụ content thay vì body, hoặc không có is_featured), hãy sửa \$fillable cho khớp đúng với migration thật của bạn — không copy máy móc.

Bước 3 — Khai báo \$casts

Thêm \$casts ngay bên dưới \$fillable để Laravel tự chuyển kiểu dữ liệu khi đọc/ghi:

```
protected $casts = [  
    'published_at' => 'datetime',  
    'is_featured' => 'boolean',  
];
```

PHÂN TÍCH

Không có \$casts: published_at trả về chuỗi string thô như “2026-06-10 08:30:00”, is_featured trả về số nguyên 0/1.

Có \$casts: published_at trả về đối tượng Carbon (gọi được ->format('d/m/Y'), ->diffForHumans(...)), is_featured trả về true/false thật trong PHP — rất quan trọng khi dùng trong điều kiện @if(\$post->is_featured).

Bước 4 — Kiểm tra Model bằng Tinker

Tinker là REPL (môi trường gõ lệnh PHP tương tác) đi kèm Laravel, dùng để test Model nhanh mà không cần qua trình duyệt:

```
php artisan tinker
```

Lần lượt gõ các lệnh sau ngay trong Tinker (nhấn Enter sau mỗi dòng):

```
>>> Post::count()  
>>> Post::first()  
>>> Post::create([  
...     'title' => 'Bài viết test từ Tinker',  
...     'slug' => 'bai-viet-test-tinker',  
...     'body' => 'Nội dung thử nghiệm.',  
...     'category_id' => 1,  
...     'user_id' => 1,  
...     'published_at' => now(),  
...     'is_featured' => true,  
... ])  
>>> Post::find(1)  
>>> exit
```

CHECKPOINT

Post::count() chạy không lỗi → kết nối Model–bảng đúng.

Post::create([...]) trả về object Post mới (có id) → \$fillable đã khai báo đúng, không bị MassAssignmentException.

Gõ lại Post::find(<id vừa tạo>)->is_featured → kết quả là true (boolean), không phải 1 → \$casts hoạt động đúng.

Bảng test case — Lab 1

#	Thao tác trong Tinker	Kết quả mong đợi
1	Post::count()	Trả về số nguyên (bằng số dòng đã seed/migrate, có thể là 0)
2	Post::create([...]) thiếu trường không có trong \$fillable, ví dụ thêm 'is_admin' => true	Trường lạ bị bỏ qua, không gây lỗi (Eloquent chỉ nhận field trong \$fillable)
3	Post::create([]) (mảng rỗng, thiếu các trường NOT NULL)	Ném lỗi QueryException — vi phạm ràng buộc NOT NULL của database, không phải lỗi Eloquent
4	Post::find(9999) (id không tồn tại)	Trả về null, không ném lỗi
5	Post::findOrFail(9999)	Ném ModelNotFoundException

2. LAB 2 — REFACTOR POSTCONTROLLER SANG ELOQUENT

Mục tiêu: thay toàn bộ logic `DB::table()` hoặc mảng dữ liệu hardcoded trong `PostController` bằng các lệnh Eloquent tương ứng, giữ nguyên hành vi `validate/flash message` đã xây dựng ở Buổi 4.

LƯU Ý

Trước khi sửa: mở `PostController.php` hiện tại, xác định đúng 5 method cần đổi: `index()`, `show()`, `store()`, `update()`, `destroy()`. Nếu Controller còn dùng mảng PHP hardcoded thay vì `DB::table()`, cách xử lý ở các bước dưới vẫn áp dụng tương tự — chỉ thay nguồn dữ liệu bằng Eloquent.

Bước 1 — `index()`: danh sách bài viết có phân trang

Trước (raw query hoặc mảng cứng):

```
// Cách cũ — ví dụ dùng Query Builder thuần
public function index()
{
    $posts = DB::table('posts')->orderBy('created_at', 'desc')->get();
    return view('posts.index', compact('posts'));
}
```

Sau (Eloquent, có phân trang):

```
use App\Models\Post;

public function index()
{
    $posts = Post::latest()->paginate(10);
    return view('posts.index', compact('posts'));
}
```

VỊ TRÍ SỬ DỤNG

`latest()` là viết tắt của `orderBy('created_at', 'desc')` — Eloquent tự hiểu cột mặc định.

`paginate(10)` tự sinh thêm các nút phân trang; trong view chỉ cần thêm `{{ $posts->links() }}` ở cuối danh sách (Blade tự render giao diện phân trang chuẩn Bootstrap nếu đã cấu hình).

Bước 2 — `show($id)`: chi tiết một bài viết

```
public function show($id)
{
    $post = Post::findOrFail($id);
    return view('posts.show', compact('post'));
}
```

PHÂN TÍCH

Dùng `findOrFail()` thay vì `find()` trong Controller: nếu id không tồn tại, Laravel tự động chuyển thành response 404 (trang lỗi mặc định hoặc `errors/404.blade.php` đã tạo ở Buổi 4) — không cần viết `if ($post === null)` thủ công.

Bước 3 — `store(Request $request)`: tạo bài viết mới

Giữ nguyên Form Request `StorePostRequest` đã xây ở Buổi 4, chỉ đổi phần lưu dữ liệu:

```
public function store(StorePostRequest $request)
{
    Post::create($request->validated());

    return redirect()->route('posts.index')
        ->with('success', 'Đã thêm bài viết mới thành công.');
```

VỊ TRÍ SỬ DỤNG

`$request->validated()` trả về mảng dữ liệu đã qua validate, đúng định dạng mà `Post::create()` cần. Vì `$fillable` đã khai báo đúng ở Lab 1, không cần liệt kê từng trường thủ công như `$post->title = $request->title; ...`

Bước 4 — `edit($id)` và `update(Request $request, $id)`

```
public function edit($id)
{
    $post = Post::findOrFail($id);
    return view('posts.edit', compact('post'));
}

public function update(UpdatePostRequest $request, $id)
{
    $post = Post::findOrFail($id);
    $post->update($request->validated());

    return redirect()->route('posts.index')
        ->with('success', 'Đã cập nhật bài viết thành công.');
```

LƯU Ý

`$post->update([...])` chỉ ghi các cột thực sự có trong mảng — cột nào không gửi lên sẽ giữ nguyên giá trị cũ. Khác với việc gán lại toàn bộ object, nên không lo mất dữ liệu các trường không có trên form sửa.

Bước 5 — `destroy($id)`: xóa bài viết

```
public function destroy($id)
{
    Post::findOrFail($id)->delete();

    return redirect()->route('posts.index')
        ->with('success', 'Đã xoá bài viết.');
}
```

Bước 6 — Cập nhật cú pháp trong View (nếu cần)

Nếu View trước đây dùng mảng/object stdClass (cú pháp \$post['title'], \$post->title từ DB::table()), kiểm tra lại — Eloquent Model trả về object nên cú pháp \$post->title vẫn đúng, nhưng \$post['title'] theo kiểu mảng sẽ KHÔNG hoạt động trừ khi Model implement ArrayAccess (Eloquent có hỗ trợ, nhưng nên dùng -> cho rõ ràng). Một điểm cần sửa chắc chắn:

```
{{-- index.blade.php — thêm dòng này cuối danh sách để hiển thị phân trang --}}
{{ $posts->links() }}
```

Bước 7 — Route Model Binding (nâng cao, không bắt buộc)

PHÂN TÍCH

Laravel có thể tự động chuyển {id} trên URL thành object Model, giúp Controller gọn hơn nữa — bỏ hẳn findOrFail():

```
// Route Model Binding — Laravel tự tìm Post theo id trong URL
public function show(Post $post)
{
    return view('posts.show', compact('post'));
}

public function edit(Post $post)
{
    return view('posts.edit', compact('post'));
}

public function update(UpdatePostRequest $request, Post $post)
{
    $post->update($request->validated());
    return redirect()->route('posts.index')->with('success', 'Đã cập nhật bài
viết thành công.');
```

```
}

public function destroy(Post $post)
{
    $post->delete();
    return redirect()->route('posts.index')->with('success', 'Đã xoá bài
viết.');
```


VỊ TRÍ SỬ DỤNG

Route Model Binding tự gọi `findOrFail()` phía sau hậu trường — nếu id không tồn tại vẫn trả 404 tự động. Đây là pattern khuyến nghị cho code sạch hơn, nhưng Lab này không bắt buộc đổi nếu thời gian không cho phép; làm đúng Bước 1–6 là đủ điểm thực hành.

Bước 8 — Test toàn bộ CRUD qua trình duyệt

1. Mở `http://127.0.0.1:8000/posts` — danh sách hiển thị đúng, có phân trang nếu trên 10 bài.
2. Bấm vào một bài viết — trang chi tiết hiển thị đúng dữ liệu.
3. Thử truy cập `/posts/9999` (id không tồn tại) — phải ra trang 404, không phải lỗi trắng trang hay exception thô.
4. Tạo bài viết mới qua form — kiểm tra flash message thành công, dữ liệu xuất hiện trong danh sách.
5. Sửa một bài viết — kiểm tra dữ liệu cập nhật đúng, các trường không sửa vẫn giữ nguyên.
6. Xóa một bài viết — kiểm tra bài biến mất khỏi danh sách, flash message hiển thị đúng.

CHECKPOINT

Toàn bộ 6 thao tác trên chạy không có lỗi PHP/SQL hiển thị ra trình duyệt; PostController không còn dòng nào gọi `DB::table('posts')` hay mảng dữ liệu hardcoded.

Bảng test case — Lab 2

#	Thao tác	Kết quả mong đợi
1	GET /posts	200 OK, danh sách bài viết kèm phân trang
2	GET /posts/{id hợp lệ}	200 OK, đúng dữ liệu bài viết
3	GET /posts/{id không tồn tại}	404 — trang errors/404.blade.php
4	POST /posts với dữ liệu hợp lệ	Redirect về index, flash message thành công, bản ghi mới trong DB
5	POST /posts thiếu trường bắt buộc	Quay lại form, hiển thị lỗi validate, dữ liệu cũ giữ qua <code>old()</code>
6	PUT /posts/{id} cập nhật một phần trường	Các trường không gửi giữ nguyên giá trị cũ
7	DELETE /posts/{id}	Redirect về index, bản ghi không còn trong danh sách
8	Kiểm tra database trực tiếp (TablePlus/phpMyAdmin) sau mỗi thao tác	Dữ liệu khớp đúng với thao tác vừa thực hiện trên giao diện

3. LAB 3 — SOFT DELETE & LARAVEL DEBUGBAR (NÂNG CAO / TỰ CHỌN)

Mục tiêu: biến thao tác xoá bài viết thành xoá mềm (có thể khôi phục), và quan sát trực tiếp số lượng query SQL mà trang index thực sự chạy — chuẩn bị nền tảng cho vấn đề N+1 sẽ giải quyết ở Buổi 7.

Bước 1 — Tạo migration thêm cột deleted_at

```
php artisan make:migration add_deleted_at_to_posts_table --table=posts
```

Mở file migration vừa tạo trong database/migrations/, hoàn thiện up() và down():

```
public function up(): void
{
    Schema::table('posts', function (Blueprint $table) {
        $table->softDeletes();
    });
}

public function down(): void
{
    Schema::table('posts', function (Blueprint $table) {
        $table->dropSoftDeletes();
    });
}
```

```
php artisan migrate
```

CHECKPOINT

Kiểm tra trong TablePlus/phpMyAdmin: bảng posts xuất hiện thêm cột deleted_at, kiểu TIMESTAMP, cho phép NULL.

Bước 2 — Thêm trait SoftDeletes vào Model

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Post extends Model
{
    use SoftDeletes;
}
```

```
protected $fillable = [
    'title', 'slug', 'body', 'category_id', 'user_id', 'published_at',
    'is_featured',
];

protected $casts = [
    'published_at' => 'datetime',
    'is_featured' => 'boolean',
];
}
```

LƯU Ý

Quên dòng `use Illuminate\Database\Eloquent\SoftDeletes;` ở đầu file là lỗi rất hay gặp — PHP sẽ báo “Class “SoftDeletes” not found” vì namespace của trait không tự động nhận diện chỉ bằng `use SoftDeletes;` bên trong class.

Bước 3 — Kiểm tra cơ chế xoá mềm bằng Tinker

```
php artisan tinker
```

```
>>> $post = Post::first()
>>> $post->delete()
>>> Post::find($post->id)
>>> Post::withTrashed()->find($post->id)
>>> exit
```

PHÂN TÍCH

`Post::find($post->id)` trả về null — Eloquent tự thêm điều kiện `WHERE deleted_at IS NULL` vào mọi câu truy vấn (Global Scope).

`Post::withTrashed()->find($post->id)` vẫn trả về object đầy đủ — dòng dữ liệu thật trong database không hề mất, chỉ bị ẩn khỏi truy vấn mặc định.

Bước 4 — Thêm route cho trang “Bài viết đã xoá”

Trong `routes/web.php`, thêm 2 route mới — đặt TRƯỚC `Route::resource('posts', PostController::class)` để tránh xung đột với route `show ({post})`:

```
Route::get('/posts/trashed', [PostController::class, 'trashed'])-
>name('posts.trashed');
Route::patch('/posts/{id}/restore', [PostController::class, 'restore'])-
>name('posts.restore');

Route::resource('posts', PostController::class);
```

LƯU Ý

Thứ tự route quan trọng: nếu `Route::resource()` khai báo trước, Laravel sẽ hiểu `/posts/trashed` là `/posts/{post}` với `post = "trashed"` và cố tìm bài viết có id `"trashed"` → lỗi 404 sai chỗ. Luôn đặt route cụ thể (literal segment) trước route có tham số động.

Bước 5 — Thêm method `trashed()` và `restore()` vào `PostController`

```
public function trashed()
{
    $posts = Post::onlyTrashed()->latest('deleted_at')->paginate(10);
    return view('posts.trashed', compact('posts'));
}

public function restore($id)
{
    Post::onlyTrashed()->findOrFail($id)->restore();

    return redirect()->route('posts.trashed')
        ->with('success', 'Đã khôi phục bài viết.');
```

Bước 6 — Tạo view `trashed.blade.php`

Tạo `resources/views/posts/trashed.blade.php`, kế thừa layout chung, lặp qua `$posts` và thêm form khôi phục cho mỗi dòng:

```
@extends('layouts.app')

@section('content')
    <h2>Bài viết đã xoá</h2>

    @forelse ($posts as $post)
        <div class="d-flex justify-content-between align-items-center border-bottom py-2">
            <div>
                <strong>{{ $post->title }}</strong>
                <div class="text-muted small">Đã xoá lúc: {{ $post->deleted_at->format('d/m/Y H:i') }}</div>
            </div>
            <form action="{{ route('posts.restore', $post->id) }}" method="POST">
                @csrf
                @method('PATCH')
                <button class="btn btn-sm btn-outline-success">Khôi phục</button>
            </form>
        </div>
    @empty
        <p>Không có bài viết nào trong thùng rác.</p>
    @endforelse

    {{ $posts->links() }}
@endsection
```

CHECKPOINT

Truy cập `/posts/trashed` → thấy đúng các bài đã xóa ở Bước 3 và Lab 2. Bấm “Khôi phục” → bài viết quay lại danh sách `/posts` bình thường.

Bước 7 — Cài đặt Laravel Debugbar

```
composer require barryvdh/laravel-debugbar --dev
```

VỊ TRÍ SỬ DỤNG

Sau khi cài, mở lại trang `/posts` trên trình duyệt — một thanh công cụ Debugbar xuất hiện ở cuối trang (chỉ hiện khi `APP_DEBUG=true` trong `.env`).

Bấm vào tab Queries — đếm số lượng câu SQL thực thi cho trang danh sách 10 bài viết. Ghi lại con số này (ví dụ: 2 query — 1 cho COUNT phân trang, 1 cho SELECT danh sách).

PHÂN TÍCH

Con số query ở bước này còn thấp vì Post chưa có quan hệ (relationship) nào được load. Ở Buổi 7, khi thêm hiển thị tên Category hoặc Tags cho mỗi bài viết mà không dùng Eager Loading, số query sẽ tăng vọt theo kiểu $N+1$ — Debugbar chính là công cụ dùng để phát hiện vấn đề đó.

Bảng test case — Lab 3

#	Thao tác	Kết quả mong đợi
1	Xoá một bài viết từ trang index	Bài biến mất khỏi <code>/posts</code> nhưng vẫn còn trong database (cột <code>deleted_at</code> có giá trị)
2	Truy cập <code>/posts/trashed</code>	Hiện thị đúng bài vừa xóa ở bước 1
3	Bấm nút Khôi phục	Bài quay lại <code>/posts</code> , biến mất khỏi <code>/posts/trashed</code>
4	<code>Post::withTrashed()->count()</code> trong Tinker	Lớn hơn hoặc bằng <code>Post::count()</code> (bao gồm cả bản ghi đã xóa mềm)
5	Mở Debugbar tab Queries trên trang <code>/posts</code>	Hiện thị danh sách câu SQL kèm thời gian thực thi từng câu

4. LỖI THƯỜNG GẶP TRONG 3 LAB & CÁCH KHẮC PHỤC

Lỗi / Thông báo	Nguyên nhân	Cách khắc phục
MassAssignmentException	Trường gửi lên không có trong \$fillable (hoặc \$fillable rỗng nhưng \$guarded chưa khai báo đúng)	Kiểm tra và bổ sung đúng tên cột vào \$fillable trong Model
ModelNotFoundException (404 không mong muốn)	Gọi findOrFail() với id không tồn tại trong CSDL	Kiểm tra lại id trên URL/form; nếu là hành vi đúng (xóa rồi truy cập lại) thì 404 là kết quả hợp lệ
Call to a member function ... on null	Dùng find() (không phải findOrFail()) rồi gọi thẳng method/thuộc tính trên kết quả có thể null	Đổi sang findOrFail(), hoặc kiểm tra if (\$post) trước khi dùng
SQLSTATE[42S22]: Column not found 'deleted_at'	Đã thêm trait SoftDeletes nhưng quên chạy migration thêm cột	Chạy lại php artisan migrate sau khi tạo migration ở Lab 3 Bước 1
Class "SoftDeletes" not found	Thiếu dòng use Illuminate\Database\Eloquent\SoftDeletes; ở đầu file Model	Thêm đúng namespace import phía trên class
Route /posts/trashed trả về lỗi tìm post id=trashed	Route::resource() khai báo trước route /posts/trashed	Di chuyển 2 route cụ thể (trashed, restore) lên TRƯỚC Route::resource()
{{ \$posts->links() }} không hiển thị gì / báo lỗi	Quên dùng paginate() (đang dùng get() hoặc all())	Đổi lại thành Post::latest()->paginate(10) ở Controller
Trang trắng / 500 khi mở Debugbar	APP_DEBUG=false trong .env, hoặc cache config cũ	Đặt APP_DEBUG=true khi đang phát triển local; chạy php artisan config:clear

5. TỔNG KẾT — 3 LAB ĐÃ HOÀN THÀNH

Lab	Trọng tâm kỹ thuật	Kỹ năng đạt được
Lab 1	make:model, \$fillable, \$casts, Tinker	Tạo và cấu hình đúng một Eloquent Model an toàn về mass assignment
Lab 2	latest(), paginate(), findOrFail(), create(), update(), delete()	Viết Controller CRUD hoàn toàn bằng Eloquent, không còn raw query
Lab 3	softDeletes(), SoftDeletes trait, withTrashed(), onlyTrashed(), restore(), Debugbar	Triển khai xóa mềm an toàn và biết cách đo lường số lượng query thực tế

CHECKLIST NỘI BÀI

1. Model Post có đầy đủ \$fillable, \$casts, trait SoftDeletes.
2. PostController không còn dòng nào dùng DB::table('posts') hoặc mảng dữ liệu hardcoded.
3. Tất cả 5 action CRUD (index, show, store, update, destroy) hoạt động đúng qua trình duyệt.
4. Tính năng “Bài viết đã xóa” (/posts/trashed) và nút khôi phục hoạt động đúng.
5. Laravel Debugbar đã cài đặt, đã quan sát và ghi nhớ số lượng query của trang index.
6. Push code lên GitHub, nhánh buoi6-lab, commit message rõ ràng theo từng bước (vd: “Lab1: tạo Post model”, “Lab2: refactor Controller sang Eloquent”, “Lab3: soft delete + debugbar”).

VỊ TRÍ SỬ DỤNG

Định hướng Buổi 7 — Eloquent Relationships: số lượng query thấy ở Debugbar (Lab 3, Bước 7) sẽ tăng mạnh khi thêm hiển thị Category/Tags cho mỗi bài viết nếu không dùng Eager Loading. Buổi 7 sẽ học with(), load() để gom các query đó lại, giải quyết triệt để vấn đề N+1.